DOCUMENTO TÉCNICO · FASE 1

# ProcSim Local

Simulador de escritorio de procesos, hilos y concurrencia para Windows

| | |
|---|---|
| **Asignatura** | Sistemas Operativos |
| **Grupo** | 10 |
| **Programa** | Ingeniería de Software |
| **Universidad** | Universidad Manuela Beltrán |
| **Integrantes** | *Por completar* |
| **Repositorio** | https://github.com/felipearenas256-dot/ProcSimLocal_SistemasOperativos_G10 |
| **Fecha** | Noviembre de 2026 |

# Contenido

1. Descripción y formulación del proyecto
2. Requerimientos funcionales y no funcionales
3. Estructura del producto y arquitectura inicial
4. Relación de tecnologías y herramientas
5. Referencias bibliográficas y fuentes técnicas

# Descripción y formulación del proyecto

## 1. Nombre del proyecto

**ProcSim Local**: simulador de escritorio de procesos, hilos y concurrencia para Windows.

El nombre une *Proc* (procesos) y *Sim* (simulador). *Local* indica que funciona en el equipo del usuario, sin internet ni servidor.

## 2. Descripción general del producto

ProcSim Local es una aplicación de escritorio para Windows que muestra de forma visual e interactiva lo que hace un sistema operativo cuando crea y administra procesos e hilos. También muestra cómo aparecen y se controlan los problemas de concurrencia.

El usuario elige un escenario (por ejemplo, "condición de carrera"), configura sus parámetros (número de hilos, prioridad, mecanismo de sincronización) y la aplicación lo ejecuta paso a paso. En pantalla se ven el diagrama de estados, el bloque de control de cada proceso (PCB), la línea de tiempo de los hilos y una bitácora de eventos. Cada ejecución se guarda en una base de datos local (SQLite) para revisarla después o exportarla a `.txt`.

Los procesos son **simulados**: el programa no crea ni controla procesos reales del equipo. En cambio, los escenarios de concurrencia usan **hilos reales** de Python (`threading`), que el sistema operativo Windows planifica como hilos nativos. Así, la condición de carrera y el interbloqueo que se ven en pantalla ocurren de verdad.

## 3. Planteamiento de la necesidad o problemática

### 3.1 Problemática

En la asignatura de Sistemas Operativos, los temas de procesos, hilos y concurrencia se estudian sobre todo en teoría: diapositivas, diagramas estáticos y ejercicios en papel. Esto genera varias dificultades:

1. **Conceptos abstractos y difíciles de visualizar.** Los estados de un proceso, el PCB o el cambio entre modo usuario y modo kernel ocurren dentro del sistema operativo y el estudiante no puede verlos.
2. **Problemas de concurrencia que no se ven a simple vista.** La condición de carrera, el interbloqueo y la inanición dependen del orden en que se ejecutan los hilos. Es difícil entender por qué ocurren y cómo se corrigen sin verlos en acción, y cuando aparecen en un programa real no siempre se repiten.
3. **Poca práctica.** Las herramientas reales para observar procesos (administradores de tareas, depuradores o comandos de consola) son complejas, muestran mucha información a la vez y no están pensadas para enseñar. Otras opciones requieren Linux, máquinas virtuales o instalaciones especiales.

4. **Dificultad del docente para demostrar en clase.** No hay una herramienta sencilla con la que el docente proyecte un ejemplo paso a paso, lo pause, lo repita y compare el resultado con y sin protección.

### 3.2 Necesidad

Se necesita una **herramienta sencilla, local y visual** que permita a estudiantes y docentes:

- Ver cómo un sistema operativo crea, administra y termina procesos e hilos.
- Reproducir los problemas de concurrencia y comparar el resultado sin protección contra el resultado con mutex o semáforo.
- Controlar la simulación (iniciar, pausar, avanzar paso a paso, cambiar la velocidad y reiniciar) para analizar cada evento.
- Funcionar en Windows sin internet, sin servidor y sin instalaciones complejas.
- Guardar el historial de las simulaciones para revisarlas y exportarlas.

## 4. Justificación de la solución

- **Aprender viendo y haciendo.** Un simulador convierte conceptos abstractos en algo observable. El estudiante cambia un parámetro y ve el efecto de inmediato, lo que refuerza lo visto en clase.
- **Problemas reales, de forma controlada.** Los escenarios de concurrencia usan hilos reales del sistema operativo. El error de la condición de carrera es real, pero el simulador lo hace repetible y lo explica.
- **Bajo costo y fácil acceso.** Usa Python y bibliotecas estándar (Tkinter, `threading`, `sqlite3`), sin licencias pagas. Se puede distribuir como un ejecutable `.exe` que funciona sin instalar nada más.
- **Pensado para Windows**, que es el sistema operativo de la mayoría de equipos de estudiantes y salas de cómputo, sin depender de máquinas virtuales.
- **Apoyo al docente.** Permite proyectar ejemplos en clase, repetirlos y exportar la bitácora como evidencia.
- **Coherencia con la asignatura.** Cada función corresponde a un tema del programa de Sistemas Operativos (ver la sección 10).

## 5. Objetivo general

Desarrollar una aplicación de escritorio para Windows que simule, de forma visual e interactiva, cómo un sistema operativo **crea y administra procesos e hilos**, y cómo aparecen y se controlan los problemas de concurrencia: **condición de carrera, interbloqueo e inanición**.

## 6. Objetivos específicos

1. Implementar un módulo de **gestión de procesos** que permita crear y finalizar procesos, consultar su PCB y animar su ciclo de vida.

2. Implementar un módulo de **hilos y modos de ejecución** que compare hilos con procesos independientes y muestre el paso de modo usuario a modo kernel en una llamada al sistema y en una interrupción.
3. Implementar un módulo de **concurrencia** con hilos reales que reproduzca la condición de carrera, la sección crítica con mutex y semáforo, el interbloqueo y la inanición.
4. Diseñar una **interfaz gráfica** con controles de simulación (iniciar, pausar, paso a paso, velocidad y reiniciar) y una explicación teórica en cada escenario.
5. Registrar cada evento en una **base de datos local SQLite** y permitir exportar la bitácora a `.txt`.
6. Validar el funcionamiento con **casos de prueba** (`unittest`) y documentar el proyecto en GitHub (README y manual de usuario).

## 7. Alcance inicial del producto

### Incluye

- Prototipo funcional de escritorio, ejecutable en Windows 10/11.
- Las 9 funciones descritas en la sección 9.
- Hilos reales de Python en los escenarios de concurrencia.
- Base de datos local (SQLite) para guardar escenarios, ejecuciones y bitácora.
- Repositorio público en GitHub, README y manual de usuario.
- Casos de prueba con capturas y análisis de resultados.

### No incluye

- Crear o modificar un kernel real: los procesos son simulados.
- Gestión de memoria, sistema de archivos ni planificación avanzada de CPU.
- Versiones web, móvil, Linux o macOS.
- Inicio de sesión, base de datos en servidor o modo multiusuario.
- Control de los procesos reales del equipo del usuario.

### Alcance de esta primera entrega

Planeación y estructura técnica: documentación, diagramas, modelo de datos, estructura de carpetas y repositorio. La implementación del código se hará en las siguientes fases (ver el README, sección "Estado actual y próximos avances").

## 8. Usuarios o beneficiarios

| Usuario | Descripción | Cómo usa el producto |
|---|---|---|
| **Estudiante** | Estudiante de Sistemas Operativos o de asignaturas afines. | Ejecuta los escenarios, cambia parámetros y observa los resultados para entender los conceptos. |

| Usuario | Descripción | Cómo usa el producto |
|---|---|---|
| **Docente** | Profesor de la asignatura. | Proyecta ejemplos en clase, los pausa y los repite paso a paso; exporta la bitácora. |
| **Beneficiario indirecto** | El programa académico. | Cuenta con un recurso de apoyo libre y reutilizable para la asignatura. |

## 9. Funcionalidades principales

El simulador ofrece **9 funciones** organizadas en 3 módulos.

### Módulo A · Gestión de procesos

| # | Función | Descripción | Resultado en pantalla |
|---|---|---|---|
| F1 | Crear y finalizar procesos | Crear procesos con nombre, prioridad y tiempo de ráfaga; listarlos y terminarlos. | Lista de procesos activos |
| F2 | Consultar el PCB | Ver el bloque de control de cada proceso: PID, estado, contador de programa, prioridad y registros. | Ficha PCB actualizada en vivo |
| F3 | Ciclo de vida | Animar las transiciones entre nuevo, listo, ejecución, bloqueado y terminado. | Diagrama de estados animado |

### Módulo B · Hilos y modos de ejecución

| # | Función | Descripción | Resultado en pantalla |
|---|---|---|---|
| F4 | Hilos de un proceso | Crear varios hilos dentro de un proceso y compararlos con procesos independientes: memoria compartida frente a pila y registros propios. | Línea de tiempo por hilo |
| F5 | Modo usuario ↔ modo kernel | Simular una llamada al sistema (por ejemplo, `read()`) y una interrupción de E/S, mostrando el cambio de modo y el regreso. | Secuencia de pasos explicada |

### Módulo C · Concurrencia

| # | Función | Descripción | Resultado en pantalla |
|---|---|---|---|
| F6 | Condición de carrera | Dos hilos incrementan un contador compartido sin protección. | Valor esperado vs. valor real |
| F7 | Sección crítica | Repetir el caso anterior usando mutex o semáforo. | Resultado correcto y comparación |
| F8 | Interbloqueo | Dos hilos retienen un recurso cada uno y esperan el del otro. | Grafo de espera circular |
| F9 | Inanición | Un hilo de baja prioridad nunca obtiene el recurso. | Tiempo de espera por hilo |

### En todos los escenarios

- **Controles:** iniciar, pausar, paso a paso, velocidad y reiniciar.
- **Bitácora** de eventos, guardada en la base de datos y exportable a `.txt`.
- **Explicación** breve del concepto teórico en pantalla.

## 10. Relación del proyecto con los conceptos de Sistemas Operativos

| Concepto de Sistemas Operativos | Dónde aparece en ProcSim Local |
|---|---|
| **Proceso** y su ciclo de vida (nuevo, listo, ejecución, bloqueado, terminado) | F1 y F3: creación, finalización y diagrama de estados animado. |
| **Bloque de control de proceso (PCB)** | F2: ficha con PID, estado, contador de programa, prioridad y registros. |
| **Planificación y cambio de contexto** | F3: despacho de listo a ejecución e interrupción de ejecución a listo, según la prioridad y la ráfaga. |
| **Hilos** (memoria compartida, pila y registros propios) | F4: hilos de un mismo proceso frente a procesos independientes. |
| **Modo usuario y modo kernel**, **llamadas al sistema** e **interrupciones** | F5: secuencia de una llamada `read()` y de una interrupción de E/S. |
| **Condición de carrera** | F6: dos hilos reales modifican un contador compartido sin protección. |
| **Sección crítica y exclusión mutua** (mutex y semáforo) | F7: `threading.Lock` y `threading.Semaphore` corrigen el resultado. |
| **Interbloqueo** (condiciones de Coffman, grafo de asignación de recursos) | F8: espera circular entre dos hilos y dos recursos, mostrada como grafo. |
| **Inanición** y prioridades | F9: un hilo de baja prioridad que no obtiene el recurso; tiempo de espera por hilo. |
| **Hilos del sistema operativo** | La aplicación usa `threading`, que en Windows crea hilos nativos planificados por el propio sistema operativo. |

# Requerimientos funcionales y no funcionales

Prioridad: **Alta** = obligatorio para el prototipo · **Media** = importante · **Baja** = deseable.

## 1. Requerimientos funcionales

### Módulo A · Gestión de procesos

| ID | Requerimiento | Función | Prioridad |
|---|---|---|---|
| RF-01 | El sistema debe permitir crear un proceso indicando nombre, prioridad y tiempo de ráfaga, y asignarle un PID único. | F1 | Alta |
| RF-02 | El sistema debe mostrar la lista de procesos activos con su PID, nombre, estado y prioridad. | F1 | Alta |
| RF-03 | El sistema debe permitir finalizar un proceso y pasarlo al estado *terminado*. | F1 | Alta |
| RF-04 | El sistema debe mostrar el PCB del proceso seleccionado (PID, estado, contador de programa, prioridad y registros) y actualizarlo en cada paso de la simulación. | F2 | Alta |
| RF-05 | El sistema debe animar las transiciones entre los estados nuevo, listo, ejecución, bloqueado y terminado, indicando el motivo de cada transición. | F3 | Alta |

### Módulo B · Hilos y modos de ejecución

| ID | Requerimiento | Función | Prioridad |
|---|---|---|---|
| RF-06 | El sistema debe permitir crear varios hilos dentro de un proceso y mostrar una línea de tiempo por hilo. | F4 | Alta |
| RF-07 | El sistema debe mostrar qué comparten los hilos de un proceso (memoria, archivos abiertos) y qué tiene cada uno (pila, registros), comparándolo con procesos independientes. | F4 | Media |
| RF-08 | El sistema debe simular una llamada al sistema ( `read()` ) y una interrupción de E/S, mostrando paso a paso el cambio de modo usuario a modo kernel y el regreso. | F5 | Alta |

### Módulo C · Concurrencia

| ID | Requerimiento | Función | Prioridad |
|---|---|---|---|
| RF-09 | El sistema debe ejecutar dos o más hilos reales que incrementen un contador compartido sin protección, y mostrar el valor esperado frente al valor real. | F6 | Alta |
| RF-10 | El sistema debe permitir repetir el escenario anterior con mutex ( `Lock` ) o semáforo ( `Semaphore` ) y comparar los resultados. | F7 | Alta |

| ID | Requerimiento | Función | Prioridad |
|---|---|---|---|
| RF-11 | El sistema debe reproducir un interbloqueo entre dos hilos y dos recursos, y mostrar el grafo de espera circular. | F8 | Alta |
| RF-12 | El sistema debe reproducir la inanición de un hilo de baja prioridad y mostrar el tiempo de espera de cada hilo. | F9 | Alta |

### Funciones comunes a todos los escenarios

| ID | Requerimiento | Prioridad |
|---|---|---|
| RF-13 | El sistema debe ofrecer un menú para elegir cualquiera de los 9 escenarios. | Alta |
| RF-14 | El sistema debe permitir configurar los parámetros del escenario (número de procesos o hilos, prioridad, ráfaga, mecanismo de sincronización y velocidad) y validar que los valores sean correctos antes de iniciar. | Alta |
| RF-15 | El sistema debe permitir iniciar, pausar, avanzar paso a paso, cambiar la velocidad y reiniciar la simulación. | Alta |
| RF-16 | El sistema debe mostrar una bitácora en pantalla con cada evento (tiempo, tipo, proceso o hilo, estado anterior y nuevo, mensaje). | Alta |
| RF-17 | El sistema debe guardar cada ejecución y sus eventos en la base de datos local SQLite. | Alta |
| RF-18 | El sistema debe permitir consultar el historial de ejecuciones guardadas. | Media |
| RF-19 | El sistema debe permitir exportar la bitácora de una ejecución a un archivo `.txt`. | Alta |
| RF-20 | El sistema debe mostrar una explicación breve del concepto teórico de cada escenario. | Media |

## 2. Requerimientos no funcionales

| ID | Tipo | Requerimiento |
|---|---|---|
| RNF-01 | Plataforma | La aplicación debe funcionar en Windows 10 y Windows 11 de 64 bits. |
| RNF-02 | Disponibilidad | Debe funcionar sin conexión a internet y sin servidor: todo se ejecuta y se guarda en el equipo local. |
| RNF-03 | Portabilidad | Debe poder distribuirse como un ejecutable `.exe` (PyInstaller) que no requiera instalar Python. |
| RNF-04 | Usabilidad | Un estudiante debe poder ejecutar un escenario en pocos pasos: elegir, configurar e iniciar. La interfaz debe estar en español. |

| ID | Tipo | Requerimiento |
|---|---|---|
| RNF-05 | Usabilidad | La interfaz debe leerse bien al proyectarse en clase (tamaño de letra y contraste adecuados). |
| RNF-06 | Rendimiento | La interfaz debe seguir respondiendo mientras corre la simulación; los hilos de la simulación no deben bloquear la ventana. |
| RNF-07 | Rendimiento | La aplicación debe iniciar en menos de 5 segundos en un equipo con los requisitos mínimos. |
| RNF-08 | Confiabilidad | El escenario de interbloqueo no debe congelar la aplicación: el sistema debe detectar la espera y permitir reiniciar. |
| RNF-09 | Confiabilidad | La simulación debe ser repetible: con los mismos parámetros debe mostrar el mismo tipo de resultado. |
| RNF-10 | Seguridad | La aplicación no debe crear, modificar ni terminar procesos reales del equipo del usuario. |
| RNF-11 | Integridad de datos | La base de datos debe usar llaves primarias, llaves foráneas y restricciones ( `CHECK` ) para mantener datos consistentes. |
| RNF-12 | Mantenibilidad | El código debe seguir el patrón Modelo-Vista-Controlador: la Vista no debe contener lógica de simulación. |
| RNF-13 | Mantenibilidad | El código debe estar documentado y seguir la guía de estilo PEP 8. |
| RNF-14 | Calidad | El motor de simulación debe tener casos de prueba con `unittest`. |
| RNF-15 | Control de versiones | El avance del proyecto debe registrarse con commits descriptivos en el repositorio de GitHub. |

# 3. Requerimientos de hardware y software

## 3.1 Para usar la aplicación

| Recurso | Mínimo | Recomendado |
|---|---|---|
| Sistema operativo | Windows 10 de 64 bits | Windows 11 de 64 bits |
| Procesador | Doble núcleo, 2 GHz | Cuatro núcleos o más |
| Memoria RAM | 4 GB | 8 GB |
| Espacio en disco | 200 MB libres | 500 MB libres |
| Pantalla | 1366 × 768 | 1920 × 1080 o proyector |

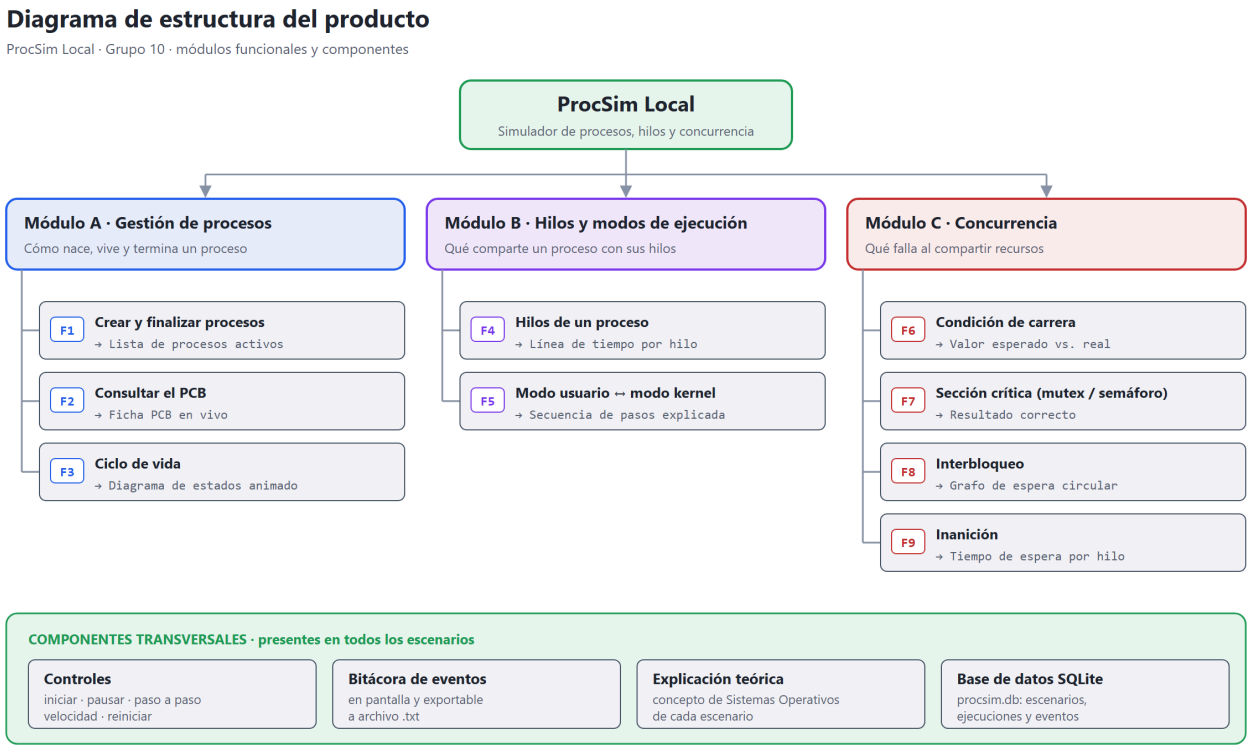
| Recurso | Mínimo | Recomendado |
|---|---|---|
| Software | Ninguno si se usa el `.exe`; Python 3.12 si se ejecuta desde el código fuente | Igual |
| Internet | No requerido | No requerido |

### 3.2 Para desarrollar

| Recurso | Uso |
|---|---|
| Python 3.12 (incluye Tkinter, `threading`, `sqlite3` y `unittest`) | Lenguaje y bibliotecas del proyecto. |
| PyInstaller | Generar el ejecutable `.exe`. |
| Visual Studio Code | Editor de código. |
| Git y GitHub | Control de versiones y repositorio público. |
| DB Browser for SQLite (opcional) | Revisar el contenido de `procsim.db`. |
| Draw.io (opcional) | Editar diagramas. |

# Estructura del producto y arquitectura inicial

## 1. Diagrama de estructura del producto



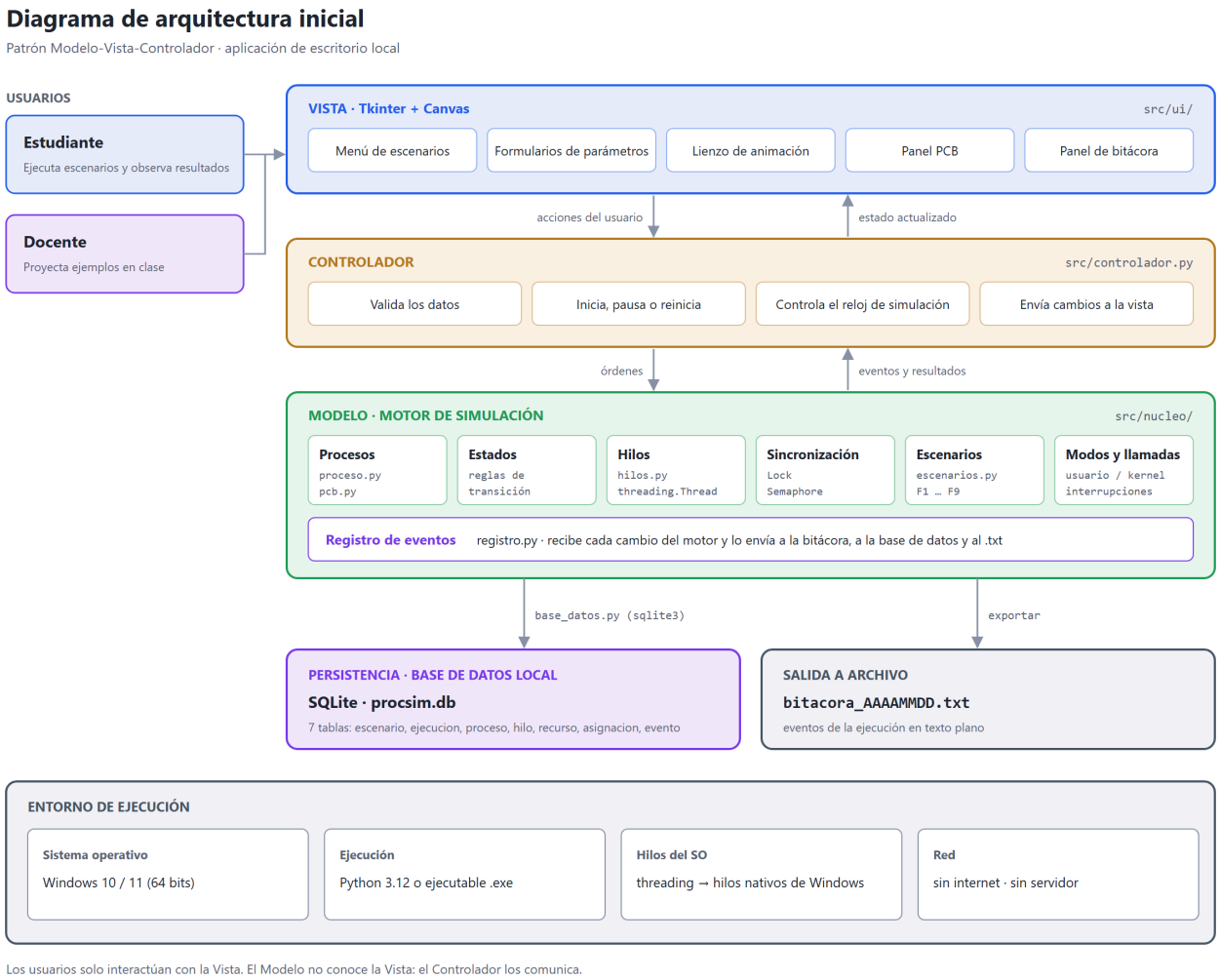

El producto se organiza en **3 módulos funcionales** con **9 funciones**. Además tiene **componentes transversales** que comparten todos los escenarios: controles de simulación, bitácora, explicación teórica y base de datos.

## 2. Módulos funcionales

| Módulo | Propósito | Funciones | Archivos principales |
|---|---|---|---|
| **A · Gestión de procesos** | Cómo nace, vive y termina un proceso. | F1 Crear y finalizar procesos · F2 Consultar el PCB · F3 Ciclo de vida | `nucleo/proceso.py`, `nucleo/pcb.py`, `ui/panel_procesos.py` |
| **B · Hilos y modos de ejecución** | Qué comparte un proceso con sus hilos y cuándo se entra al kernel. | F4 Hilos de un proceso · F5 Modo usuario ↔ modo kernel | `nucleo/hilos.py`, `nucleo/escenarios.py`, `ui/panel_procesos.py` |

| Módulo | Propósito | Funciones | Archivos principales |
|---|---|---|---|
| **C · Concurrencia** | Qué falla cuando varios hilos comparten recursos y cómo se corrige. | F6 Condición de carrera · F7 Sección crítica · F8 Interbloqueo · F9 Inanición | `nucleo/sincronizacion.py`, `nucleo/escenarios.py`, `ui/panel_concurrencia.py` |

## 3. Diagrama de arquitectura inicial




Los usuarios solo interactúan con la Vista. El Modelo no conoce la Vista: el Controlador los comunica.

La aplicación sigue el patrón **Modelo-Vista-Controlador (MVC)**.

## 4. Componentes principales del sistema

| Componente | Capa | Responsabilidad | Ubicación |
|---|---|---|---|
| Punto de entrada | — | Crea la ventana principal y arranca el controlador. | `src/main.py` |

| Componente | Capa | Responsabilidad | Ubicación |
|---|---|---|---|
| Ventana principal | Vista | Menú de escenarios, controles de simulación y panel de bitácora. | `src/ui/ventana_principal.py` |
| Panel de procesos | Vista | Lista de procesos, ficha PCB, diagrama de estados, línea de tiempo por hilo y secuencia usuario/kernel. | `src/ui/panel_procesos.py` |
| Panel de concurrencia | Vista | Contador compartido, comparación con mutex o semáforo, grafo de espera y tiempos de espera. | `src/ui/panel_concurrencia.py` |
| Controlador | Controlador | Valida los datos, inicia, pausa o reinicia, controla el reloj de simulación y envía los cambios a la vista. | `src/controlador.py` |
| Proceso y PCB | Modelo | Representan un proceso simulado y su bloque de control. | `src/nucleo/proceso.py`, `src/nucleo/pcb.py` |
| Hilos | Modelo | Crean hilos reales con `threading.Thread`. | `src/nucleo/hilos.py` |
| Sincronización | Modelo | Ofrecen los mecanismos ninguno, mutex (`Lock`) y semáforo (`Semaphore`). | `src/nucleo/sincronizacion.py` |
| Escenarios | Modelo | Definen la lógica de cada una de las 9 funciones. | `src/nucleo/escenarios.py` |
| Registro de eventos | Modelo | Recibe cada cambio del motor y lo envía a la bitácora, a la base de datos y al `.txt`. | `src/nucleo/registro.py` |
| Acceso a datos | Persistencia | Crea la base de datos y guarda o consulta ejecuciones y eventos. | `src/nucleo/base_datos.py` |
| Esquema de la base de datos | Persistencia | Tablas, llaves y restricciones. | `src/bd/esquema.sql` |

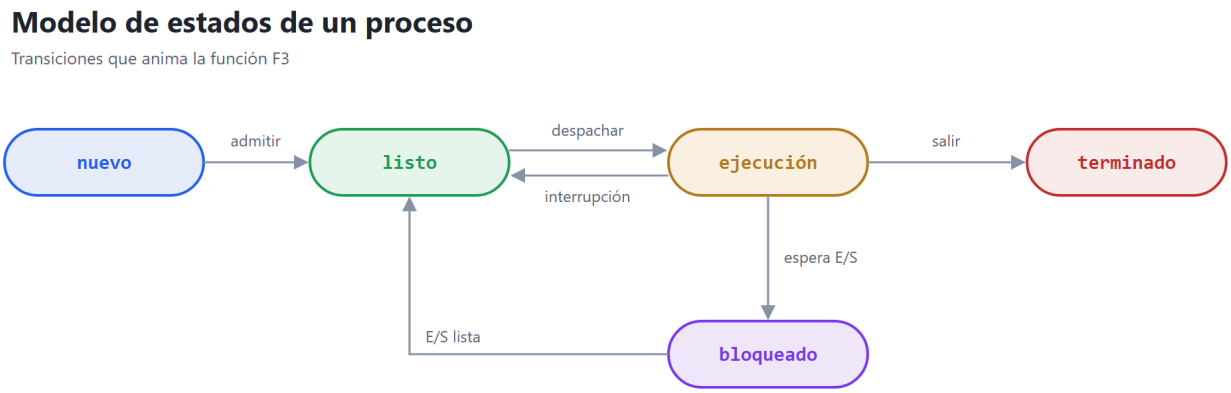
### Flujo de una simulación

1. El usuario elige un escenario y sus parámetros en la **Vista**.
2. El **Controlador** valida los datos y le pide al **Modelo** que prepare el escenario.
3. El Modelo crea los procesos, los PCB y los hilos. Cada cambio de estado genera un **evento**.
4. El **Registro de eventos** guarda el evento en la base de datos y lo entrega al Controlador.
5. El Controlador envía el cambio a la Vista, que actualiza la animación, la ficha PCB y la bitácora.
6. Al terminar, se guarda el resultado esperado y el real, y el usuario puede exportar la bitácora a `.txt`.

### Decisiones técnicas

- **La interfaz no se congela.** Tkinter solo puede actualizarse desde el hilo principal. Los hilos de la simulación ponen sus eventos en una cola (`queue.Queue`) y la Vista la revisa periódicamente con `after()`.
- **Condición de carrera repetible.** En CPython, `contador += 1` casi nunca falla por sí solo. Para que el error se vea siempre, cada hilo lee el valor, hace una pausa corta (`time.sleep`) y luego escribe. Esa pausa fuerza el cambio de contexto entre la lectura y la escritura, que es justo donde ocurre la carrera.
- **Interbloqueo sin congelar la aplicación.** Los hilos piden el segundo recurso con `acquire(timeout=...)`. Si se vence el tiempo, el sistema registra el interbloqueo, dibuja el grafo de espera y libera los recursos para poder reiniciar.

## 5. Modelo de estados de un proceso



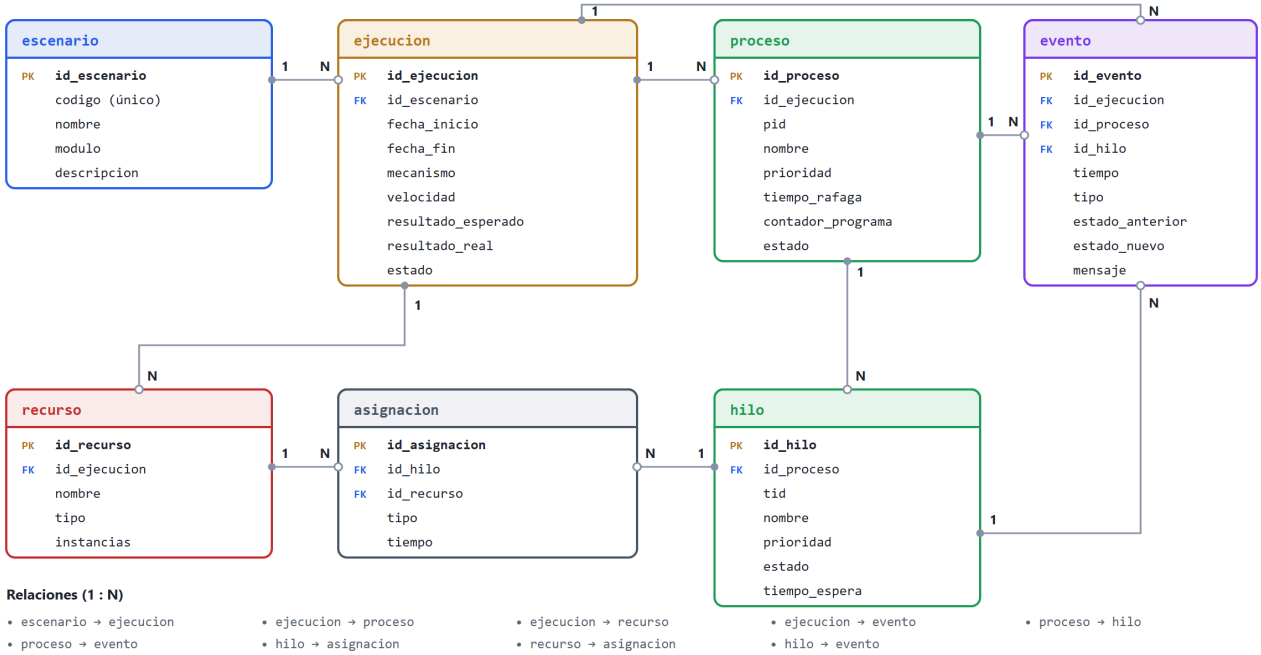

| Transición | Motivo |
|---|---|
| nuevo → listo | **admitir**: el proceso fue creado y entra a la cola de listos. |
| listo → ejecución | **despachar**: el planificador le asigna la CPU. |
| ejecución → listo | **interrupción**: se acaba su tiempo o llega uno de mayor prioridad. |
| ejecución → bloqueado | **espera E/S**: pidió una operación de entrada/salida. |
| bloqueado → listo | **E/S lista**: terminó la operación de entrada/salida. |
| ejecución → terminado | **salir**: completó su ráfaga o fue finalizado. |

## 6. Base de datos y modelo relacional

La base de datos es **SQLite**: un solo archivo local (`procsim.db`), sin servidor. El módulo `sqlite3` viene incluido en Python. El esquema completo está en `src/bd/esquema.sql`.

### Modelo relacional · base de datos local (SQLite)

src/bd/esquema.sql · PK = llave primaria · FK = llave foránea




| Tabla | Qué guarda | Relaciones |
|---|---|---|
| `escenario` | Catálogo de las 9 funciones (F1 a F9) y su módulo. | 1 escenario → N ejecuciones |
| `ejecucion` | Cada vez que se corre un escenario: fecha, mecanismo, velocidad, resultado esperado y real. | 1 ejecución → N procesos, recursos y eventos |
| `proceso` | Procesos simulados con los datos de su PCB. | 1 proceso → N hilos y eventos |
| `hilo` | Hilos de cada proceso, con prioridad y tiempo de espera. | 1 hilo → N asignaciones y eventos |
| `recurso` | Recursos compartidos (variable, dispositivo, archivo). | 1 recurso → N asignaciones |
| `asignacion` | Qué hilo **retiene** o **espera** qué recurso; con esto se dibuja el grafo de interbloqueo. | N : 1 con hilo y con recurso |
| `evento` | La bitácora: cada evento con tiempo, tipo, estados y mensaje. | N : 1 con ejecución, proceso e hilo |

## 7. Organización de archivos y directorios

```
ProcSimLocal_SistemasOperativos_G10/
├── README.md                      # presentación del proyecto
├── LICENSE                        # licencia MIT
├── documentacion/                 # documentación técnica
│   ├── descripcion_proyecto.md / .pdf
│   ├── requerimientos.md / .pdf
│   ├── arquitectura_inicial.md / .pdf
│   ├── tecnologias_herramientas.md / .pdf
│   ├── referencias.md / .pdf
│   ├── documento_tecnico.pdf      # documento consolidado para la entrega
│   └── diagramas/                 # diagramas en .svg y .png
├── src/                           # código fuente
│   ├── main.py                    # punto de entrada
│   ├── controlador.py             # CONTROLADOR
│   ├── ui/                        # VISTA (Tkinter)
│   │   ├── ventana_principal.py
│   │   ├── panel_procesos.py
│   │   └── panel_concurrencia.py
│   ├── nucleo/                    # MODELO (motor de simulación)
│   │   ├── proceso.py
│   │   ├── pcb.py
│   │   ├── hilos.py
│   │   ├── sincronizacion.py
│   │   ├── escenarios.py
│   │   ├── registro.py
│   │   └── base_datos.py
│   └── bd/
│       └── esquema.sql            # modelo relacional (SQLite)
├── recursos/
│   └── imagenes/                  # capturas e íconos
└── pruebas/                       # casos de prueba (unittest)
```

## 8. Sistema operativo y entorno de ejecución previsto

| Aspecto | Definición |
|---|---|
| Sistema operativo | Windows 10 / 11 de 64 bits. |
| Entorno | Python 3.12 durante el desarrollo; ejecutable `.exe` (PyInstaller) para los usuarios. |
| Hilos | `threading` crea hilos nativos de Windows, planificados por el sistema operativo. |
| Datos | Archivo local `procsim.db` (SQLite) en la carpeta de la aplicación. |
| Red | No requiere internet ni servidor. |

# Relación de tecnologías y herramientas

## 1. Tecnologías del producto

| Capa | Tecnología | Versión | Uso en el proyecto | Por qué se eligió |
|---|---|---|---|---|
| Lenguaje | Python | 3.12 | Lenguaje principal de la aplicación. | Es sencillo de leer, se usa en la academia y trae en su biblioteca estándar todo lo que el proyecto necesita. |
| Interfaz (front end) | Tkinter + Canvas | Incluido en Python | Ventanas, formularios, controles y animaciones (diagrama de estados, línea de tiempo, grafo de espera). | No requiere instalación adicional y `Canvas` permite dibujar y animar figuras. |
| Concurrencia | `threading` (`Thread`, `Lock`, `Semaphore`) | Incluido en Python | Hilos reales y mecanismos de sincronización de los escenarios F4 y F6 a F9. | En Windows crea hilos nativos del sistema operativo, así los problemas de concurrencia son reales. |
| Comunicación entre hilos | `queue.Queue` | Incluido en Python | Pasar los eventos de los hilos de simulación a la interfaz. | Es segura para hilos y evita que la interfaz se congele. |
| Base de datos | SQLite (`sqlite3`) | 3.x, incluido en Python | Guardar escenarios, ejecuciones, procesos, hilos, recursos y eventos. | Es una base de datos relacional en un solo archivo local, sin servidor. |
| Pruebas | `unittest` | Incluido en Python | Casos de prueba del motor de simulación. | Es el marco de pruebas estándar de Python. |
| Empaquetado | PyInstaller | 6.x | Generar el ejecutable `.exe` para Windows. | Permite usar la aplicación sin instalar Python. |

## 2. Herramientas de trabajo

| Herramienta | Uso |
|---|---|
| Git | Control de versiones local y commits. |
| GitHub | Repositorio público del proyecto, historial de commits y documentación. |
| Visual Studio Code | Editor de código, con la extensión de Python. |
| DB Browser for SQLite | Revisar las tablas y los datos de `procsim.db`. |
| Draw.io | Edición de diagramas. |
| Microsoft Word / PDF | Documento técnico de la entrega. |

## 3. Entorno de ejecución

| Aspecto | Definición |
|---|---|
| Sistema operativo | Windows 10 / 11 de 64 bits. |
| Modo de ejecución | Desde el código ( `python src/main.py` ) o con el ejecutable `.exe` . |
| Conexión | Sin internet y sin servidor. |
| Virtualización | No es necesaria: la aplicación corre directamente en Windows. |

# Referencias bibliográficas y fuentes técnicas

Formato: APA 7.ª edición.

## 1. Libros y artículos de Sistemas Operativos


- Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). *Operating systems: Three easy pieces*. Arpaci-Dusseau Books. https://pages.cs.wisc.edu/~remzi/OSTEP/
- Coffman, E. G., Elphick, M., & Shoshani, A. (1971). System deadlocks. *ACM Computing Surveys, 3*(2), 67–78. https://doi.org/10.1145/356586.356588
- Downey, A. B. (2016). *The little book of semaphores* (2.ª ed.). Green Tea Press. https://greenteapress.com/wp/semaphores/
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.
- Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.). Pearson.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4.ª ed.). Pearson.


## 2. Arquitectura de software


- Krasner, G. E., & Pope, S. T. (1988). A cookbook for using the model-view controller user interface paradigm in Smalltalk-80. *Journal of Object-Oriented Programming, 1*(3), 26–49.


## 3. Documentación técnica


- Microsoft. (s. f.). *Processes and threads*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/procthread/processes-and-threads
- PyInstaller Development Team. (s. f.). *PyInstaller manual*. https://pyinstaller.org/en/stable/
- Python Software Foundation. (s. f.). *queue — A synchronized queue class*. https://docs.python.org/3/library/queue.html
- Python Software Foundation. (s. f.). *sqlite3 — DB-API 2.0 interface for SQLite databases*. https://docs.python.org/3/library/sqlite3.html
- Python Software Foundation. (s. f.). *threading — Thread-based parallelism*. https://docs.python.org/3/library/threading.html
- Python Software Foundation. (s. f.). *tkinter — Python interface to Tcl/Tk*. https://docs.python.org/3/library/tkinter.html
- Python Software Foundation. (s. f.). *unittest — Unit testing framework*. https://docs.python.org/3/library/unittest.html
- SQLite. (s. f.). *SQLite documentation*. https://www.sqlite.org/docs.html